

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**По лабораторной работе №2**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: Кратчайшие пути в графе: коммивояжёр.**

Студент гр. 4345

Беденков А.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

### Входные данные:

- $n$  — количество городов ( $5 \leq n \leq 15$ ).
- Матрица расстояний между городами размером  $n \times n$ , где  $\text{graph}[i][j]$  обозначает расстояние от города  $i$  до города  $j$ . Если  $\text{graph}[i][j]=0$  (и  $i \neq j$ ), это означает, что прямого пути между городами нет.

### Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

### Sample Input 1:

```
5
0 1 13 23 7
12 0 15 18 28
21 29 0 33 28
23 19 34 0 38
5 40 7 39 0
```

### Sample Output 1:

```
78
0 4 2 3 1 0
```

### Sample Input 2:

```
3
0 1 0
```

1 0 1

0 1 0

**Sample Output 2:**

no path

Вариант 7:

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik.

Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

## Выполнение работы

Программа реализует два различных подхода к решению классической задачи коммивояжёра: точный алгоритм на основе динамического программирования с использованием битовых масок (известный как алгоритм Беллмана-Хелда-Карпа) и приближённый эвристический алгоритм (жадная стратегия, АЛШ-1). Оба алгоритма принимают на вход матрицу смежностей, формируют маршруты и выводят результаты в соответствии с требованиями.

Для обеспечения гибкости программа была модернизирована: она поддерживает как ввод данных из текстового файла (через передачу аргумента командной строки), так и чтение из стандартного потока ввода `stdin`.

### Точный алгоритм (Хелда-Карпа)

#### Основные структуры данных

1. **Двумерный список** `graph` – матрица смежности графа размером  $n \times n$ . Элемент `graph[i][j]` содержит вес направленного ребра из вершины  $i$  в вершину  $j$ . Значение 0 означает отсутствие прямого пути между вершинами. Данная структура заполняется на этапе парсинга входных данных.
2. **Двумерный список** `memo` – таблица кэширования для динамического программирования размером  $(1 \ll n) \times n$ . Индексы таблицы интерпретируются следующим образом: первый индекс `mask` представляет битовую маску уже посещенных вершин, второй индекс `u` — текущую вершину, в которой находится коммивояжер. Значение `memo[mask][u]` хранит минимальную стоимость завершения маршрута из данного состояния. Первоначально массив заполнен значениями -1, что является индикатором невычисленного состояния.
3. **Двумерный список** `nxt_node` – структура размером  $(1 \ll n) \times n$ , предназначенная для последующего восстановления оптимального пути. Элемент `nxt_node[mask][u]` сохраняет номер вершины  $v$ , переход в

которую из состояния  $(mask, u)$  дает минимальную итоговую стоимость маршрута.

### Описание алгоритма

Алгоритм реализует парадигму динамического программирования «сверху вниз» (рекурсия с мемоизацией). На вход алгоритм получает количество вершин графа  $n$  и матрицу смежности.

Работа точного алгоритма начинается с вызова функции  $tsp\_dp(mask, u)$  с начальными аргументами  $mask = 1$  (посещена только стартовая вершина 0, что в двоичном виде равно  $0...001$ ) и  $u = 0$  (текущая вершина).

Внутри рекурсивной функции  $tsp\_dp$  выполняются следующие шаги:

1. **Базовый случай:** Производится проверка  $mask == (1 \ll n) - 1$ . Данное условие означает, что все биты с 0 по  $n - 1$  установлены в единицу, то есть все вершины графа были посещены. В этом случае алгоритм проверяет наличие замыкающего ребра из текущей вершины  $u$  в стартовую 0. Если  $graph[u][0] > 0$ , функция возвращает вес этого ребра; если ребра нет — возвращается  $float('inf')$  (бесконечность).
2. **Проверка кэша:** Если  $memo[mask][u] \neq -1$ , это означает, что оптимальный путь из данного подмножества уже был вычислен ранее. Функция немедленно возвращает сохраненное значение, избегая дублирования вычислений.
3. **Переход состояния:** Если значение не вычислено, алгоритм инициализирует локальную переменную  $ans$  бесконечностью. Затем в цикле перебираются все возможные следующие вершины  $v$  от 0 до  $n - 1$ .
4. **Валидация перехода:** Переход в вершину  $v$  считается допустимым, если она еще не посещена (проверяется побитовым «И»:  $not (mask \& (1 \ll v))$ ) и существует ребро  $graph[u][v] > 0$ .
5. **Рекурсивный вызов:** Для каждого валидного кандидата вычисляется стоимость  $res = graph[u][v] + tsp\_dp(mask | (1 \ll v), v)$ . Операция

побитового «ИЛИ»  $\text{mask} \mid (1 \ll v)$  обновляет маску, пометая вершину  $v$  как посещенную.

6. **Мемоизация:** Из всех кандидатов выбирается тот, который дает минимальное значение  $\text{res}$ . Это значение записывается в  $\text{memo}[\text{mask}][u]$ , а номер выбранной вершины  $v$  сохраняется в  $\text{nxt\_node}[\text{mask}][u]$ .

После завершения рекурсии в функции  $\text{solve()}$  проверяется полученный результат. Если стоимость равна бесконечности, выводится сообщение "no path". В противном случае происходит восстановление пути: начиная с  $\text{mask} = 1$  и  $u = 0$ , алгоритм в цикле  $\text{while}$  считывает следующую вершину из  $\text{nxt\_node}[\text{mask}][u]$ , добавляет её в результирующий список  $\text{path}$  и обновляет  $\text{mask}$  и  $u$ , пока не будут посещены все вершины. В конце в путь добавляется возвращение в вершину 0.

### Оптимизации

1. **Использование битовых масок:** Представление множества посещенных вершин в виде одного целого числа (маски) позволяет выполнять операции добавления элемента в множество и проверки принадлежности за  $O(1)$  с помощью побитовых операций сдвига, И и ИЛИ. Это многократно быстрее, чем использование массивов или хеш-таблиц.
2. **Мемоизация результатов:** Классический перебор имеет сложность  $O(n!)$ , что делает задачу нерешаемой для  $n \geq 12$ . Таблица мемо сохраняет результаты вычисленных подзадач. Поскольку количество уникальных комбинаций (подмножество посещенных вершин + текущая вершина) ограничено, мемоизация снижает сложность до экспоненциальной.
3. **Раннее отсечение:** Условие  $\text{graph}[u][v] > 0$  предотвращает рекурсивный спуск в ветви дерева решений, где физически отсутствует путь между городами, что сильно сокращает константу времени выполнения на разреженных графах.

### Оценка сложности (Точный алгоритм)

- **Временная сложность:** Оценивается количеством уникальных состояний, умноженным на время обработки одного состояния. Состояние описывается парой (mask, u). Маска может принимать  $2^n$  значений, а u может принимать n значений. Итого  $n \cdot 2^n$  состояний. В каждом состоянии выполняется цикл по n вершинам. Следовательно, общая временная сложность составляет  $O(n^2 \cdot 2^n)$ . Для  $n = 15$  это составляет около  $7.3 \cdot 10^6$  операций, что выполняется за доли секунды.
- **Пространственная сложность:** Затраты памяти определяются двумя таблицами мемоизации memo и nxt\_node, каждая из которых имеет размер  $2^n \times n$ . Глубина рекурсии составляет n. Общая пространственная сложность равна  $O(n \cdot 2^n)$ .

### Приближенный алгоритм (АЛШ-1 / Жадный метод)

#### Основные структуры данных

1. **Список visited** – одномерный массив логических значений размером n. visited[i] = True, если вершина i уже включена в маршрут.
2. **Список path** – хранит последовательность вершин в порядке их добавления в строящийся маршрут.
3. **Переменные состояния:** current\_city (текущая вершина), total\_cost (накопленная сумма весов ребер).

#### Описание

#### алгоритма

Данный алгоритм реализует простую жадную эвристику: на каждом шаге коммивояжер выбирает ближайший к нему непосещенный город.

1. Алгоритм стартует из вершины 0. Устанавливается visited[0] = True, вершина добавляется в path.
2. Запускается цикл, который должен выполняться  $n - 1$  раз (для оставшихся вершин).
3. На каждой итерации инициализируется переменная min\_dist = float('inf') и best\_next\_city = -1.

4. Во внутреннем цикле перебираются все вершины  $v$ . Если вершина  $v$  не посещена (`not visited[v]`) и в неё ведет путь из текущей вершины (`graph[current_city][v] > 0`), её вес сравнивается с `min_dist`.
5. Если вес меньше текущего минимума,  $v$  запоминается как лучший кандидат.
6. После проверки всех соседей алгоритм проверяет, был ли найден кандидат. Если `best_next_city == -1`, это означает, что мы попали в "тупик" и построить гамильтонов цикл жадным методом из этой точки невозможно. Алгоритм завершается, возвращая бесконечность.
7. Если вершина найдена, она помечается как посещенная, добавляется в `path`, `total_cost` увеличивается на вес ребра, а `current_city` обновляется.
8. По завершении цикла проверяется возможность возврата в стартовую вершину: если `graph[current_city][0] > 0`, цикл замыкается. Иначе возвращается отсутствие пути.

#### **Оценка сложности (Приближенный алгоритм)**

- **Временная сложность:** Внешний цикл выполняется  $n - 1$  раз. На каждой итерации внутренний цикл перебирает  $n$  вершин для поиска минимума. Таким образом, выполняется порядка  $n^2$  сравнений. Временная сложность строго равна  $O(n^2)$ .
- **Пространственная сложность:** Основную память занимает исходная матрица смежности ( $n^2$ ). Списки `visited` и `path` занимают  $O(n)$ . Общая пространственная сложность  $O(n^2)$ .



## Тестирование

Тестирование производилось локально на сгенерированных графах различных размеров. Программа поддерживает два способа ввода данных: автоматическое чтение из текстового файла (если имя файла передано при запуске) и стандартный ручной ввод с клавиатуры.

### *Тест 1 – Стандартная полная матрица размера 5*

*Ввод:*

5

0 1 13 23 7

12 0 15 18 28

21 29 0 33 28

23 19 34 0 38

5 40 7 39 0

*Вывод:*

*АЛШ-1 (Приближенный метод)*

106

0 1 2 4 3 0

*ДП (Точный метод)*

78

0 4 2 3 1 0

### *Тест 2 – Матрица без возможного гамильтонова цикла*

*Ввод:*

3

0 1 0

1 0 1

0 1 0

*Вывод:*

*АЛШ-1 (Приближенный метод)*

*по path*

*ДП (Точный метод)*

*по path*

***Тест 3 – Изолированный граф (отсутствие ребер)***

*Ввод:*

5

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

0 0 0 0 0

*Вывод:*

*АЛШ-1 (Приближенный метод)*

*по path*

*ДП (Точный метод)*

*по path*

***Тест 4 – Случайная полная матрица максимального размера (N=15)***

*Для тестирования был использован написанный генератор матриц*

*Ввод:*

15

0 29 17 49 27 0 23 31 25 29 30 18 30 38 42

19 0 0 22 0 18 6 26 17 4 30 21 12 49 0

3 23 0 27 30 2 0 27 16 49 26 30 5 38 34

30 48 10 0 6 37 16 17 20 29 44 13 1 17 10

11 14 0 7 0 26 31 50 27 11 36 21 1 36 42

48 12 22 48 42 0 44 43 1 34 25 8 21 0 50

16 38 6 9 25 0 0 12 28 42 1 29 44 0 9  
30 7 37 0 0 31 10 0 48 2 43 2 25 14 50  
29 0 35 40 0 5 19 26 0 42 5 50 3 35 31  
16 18 43 20 39 49 13 28 27 0 8 1 0 49 31  
0 0 48 32 44 23 30 0 34 4 0 35 11 10 29  
0 8 7 31 29 28 0 40 23 20 5 0 17 16 32  
10 42 14 0 46 19 4 12 3 9 9 20 0 38 25  
10 39 9 14 27 39 39 33 30 43 20 10 15 0 15  
34 14 13 33 35 15 0 6 22 26 42 11 19 44 0

*Вывод:*

*АЛШ-1 (Приближенный метод)*

156

0 2 5 8 12 6 10 9 11 1 3 4 13 14 7 0

*ДП (Точный метод)*

99

0 2 5 8 10 13 14 7 9 11 1 6 3 4 12 0

## ***Вывод***

В ходе выполнения лабораторной работы были успешно реализованы и проанализированы два алгоритма решения задачи коммивояжёра: точный метод на основе динамического программирования с битовыми масками (алгоритм Хелда-Карпа) и приближённый эвристический алгоритм локального поиска (АЛШ-1).

Точный алгоритм обеспечивает нахождение стопроцентно оптимального гамильтонова цикла за полиномиально-экспоненциальное время  $O(n^2 \cdot 2^n)$  и гарантирует корректный результат для графов размером до 15 вершин включительно. Анализ показал, что применение битовых масок и мемоизации является критически важным: без них алгоритм скатился бы до чистого перебора  $O(n!)$  и завис бы на тестах с  $N > 11$ . Однако применение метода ограничено физическими возможностями оперативной памяти для хранения  $2^n$  состояний при увеличении  $n$ .

Приближённый алгоритм, работающий за полиномиальное время  $O(n^2)$ , позволяет строить допустимые маршруты практически мгновенно для графов любой размерности. Тем не менее, тестирование явно показало главный недостаток жадной стратегии: на средних и больших графах алгоритм делает локально выгодные шаги в начале, что часто приводит к необходимости прохождения длинных, невыгодных ребер в конце для замыкания цикла, тем самым не гарантируя оптимальности итогового решения.

Обе реализации оформлены в единый программный код с применением гибкого чтения данных (поток ввода/аргументы командной строки). Проведенное тестирование на различных наборах входных данных подтвердило корректность работы логики.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

**main.py**

```
import sys

sys.setrecursionlimit(2000)

def alsh1_approximate(graph, n):
    visited = [False] * n
    current_city = 0
    visited[current_city] = True
    path = [current_city]
    total_cost = 0

    for _ in range(n - 1):
        best_next_city = -1
        min_dist = float('inf')

        for v in range(n):
            if not visited[v] and graph[current_city][v] > 0:
                if graph[current_city][v] < min_dist:
                    min_dist = graph[current_city][v]
                    best_next_city = v

        if best_next_city == -1:
            return float('inf'), []

        visited[best_next_city] = True
        path.append(best_next_city)
        total_cost += min_dist
        current_city = best_next_city
```

```

    if graph[current_city][0] > 0:
        total_cost += graph[current_city][0]
        path.append(0)
        return total_cost, path
    else:
        return float('inf'), []

def solve():
    data = (open(sys.argv[1]).read() if len(sys.argv) > 1 else
sys.stdin.read()).split()

    if not data: return

    n = int(data[0])

    graph = [[int(data[i*n + j + 1]) for j in range(n)] for i in
range(n)]

    print(" АЛШ-1 (Приближенный метод) ")
    approx_cost, approx_path = alsh1_approximate(graph, n)
    if approx_cost == float('inf'):
        print("no path")
    else:
        print(approx_cost)
        print(" ".join(map(str, approx_path)))
    print()

    print(" ДП (Точный метод) ")
    memo = [[-1] * n for _ in range(1 << n)]
    nxt_node = [[-1] * n for _ in range(1 << n)]

    def tsp_dp(mask, u):
        if mask == (1 << n) - 1:
            if graph[u][0] > 0:
                return graph[u][0]

```

```

        return float('inf')

    if memo[mask][u] != -1:
        return memo[mask][u]

    ans = float('inf')
    best_nxt = -1

    for v in range(n):
        if not (mask & (1 << v)) and graph[u][v] > 0:
            res = graph[u][v] + tsp_dp(mask | (1 << v), v)
            if res < ans:
                ans = res
                best_nxt = v

    memo[mask][u] = ans
    nxt_node[mask][u] = best_nxt
    return ans

min_cost = tsp_dp(1, 0)

if min_cost == float('inf'):
    print("no path")
else:
    print(min_cost)

path = [0]
mask = 1
u = 0
while mask != (1 << n) - 1:
    v = nxt_node[mask][u]
    path.append(v)

```

```

        mask |= (1 << v)

    u = v

    path.append(0)

    print(" ".join(map(str, path)))

if __name__ == '__main__':
    solve()

```

## loader.py

```

import random

n = int(input("Количество городов: "))

name = input("Имя файла (Enter для input.txt): ") or "input.txt"

with open(name, 'w') as f:
    f.write(f"{n}\n")

    f.write("\n".join(" ".join(str(random.randint(1, 50) if i != j and
random.random() > 0.1 else 0) for j in range(n)) for i in range(n)))

```